· Каисс сдвиг строк распределяет в каждый столбец по 1 бо
с типом A

· Как и в первом раунде, если в столбце 1 байт A, а остальные C, то
байты в столбце преобразуются в A.

AAAA
AAAA → После 2-го раунда
AAAA
AAAA

SSSS
SSSS → После 3-го раунда
SSSS
SSSS

Т.к. в последнем раунде не происходит преобразование столбц
(спецификация AES), а остальные преобразования переводят A в A, то для 4-раун
AES в результате последнего раунда не происходит изменение значени
интеграла вплоть до ⊕ с раундовыми ключами. В таком случае все, что
это для каждого байта предположить значение соответствующего
байта раундового ключа, получить предположительный текст 3-го р
и проверить равен ли интеграл от соответствующего блока нулю. Если р
байт раундового ключа можно считать найденым.

Схему можно расширить до семираундовой рассматривая, от чего
зависит преобразование интеграла от конкретного блока.

Slide - атака

Условие проведения: одинаковые раундовые ключи, одинаковая раундовая ф-я



Алгоритм:

1) Возьмём пару (о.т.; ш.т.) (X, y)

2) Берём пару (о.т.; ш.т.) (X', y'), отличную от (X, y) ~~отличную от (X, y)~~ ~~$X' = F(X)$ $y' = F(y)$~~

3) Предположим, что $X' = F(X)$, $y' = F(y)$, т.е. X' и y' являются результатом раундового шифрования X и y. $((X, y); (X', y'))$ - „сдвиговая пара"

Найдём K_{First} по отношению к $X' = F(X)$ и $K_{last} - y' = F(y)$

4) Проверяем $K = K_{First} = K_{last}$, по условию все раундовые ключи одинаковые ра-во должно выполняться. Если это неверно, то возвращаемся к пункту 2 и рассматриваем новую пару

5) Проверяем K, подставляя в F

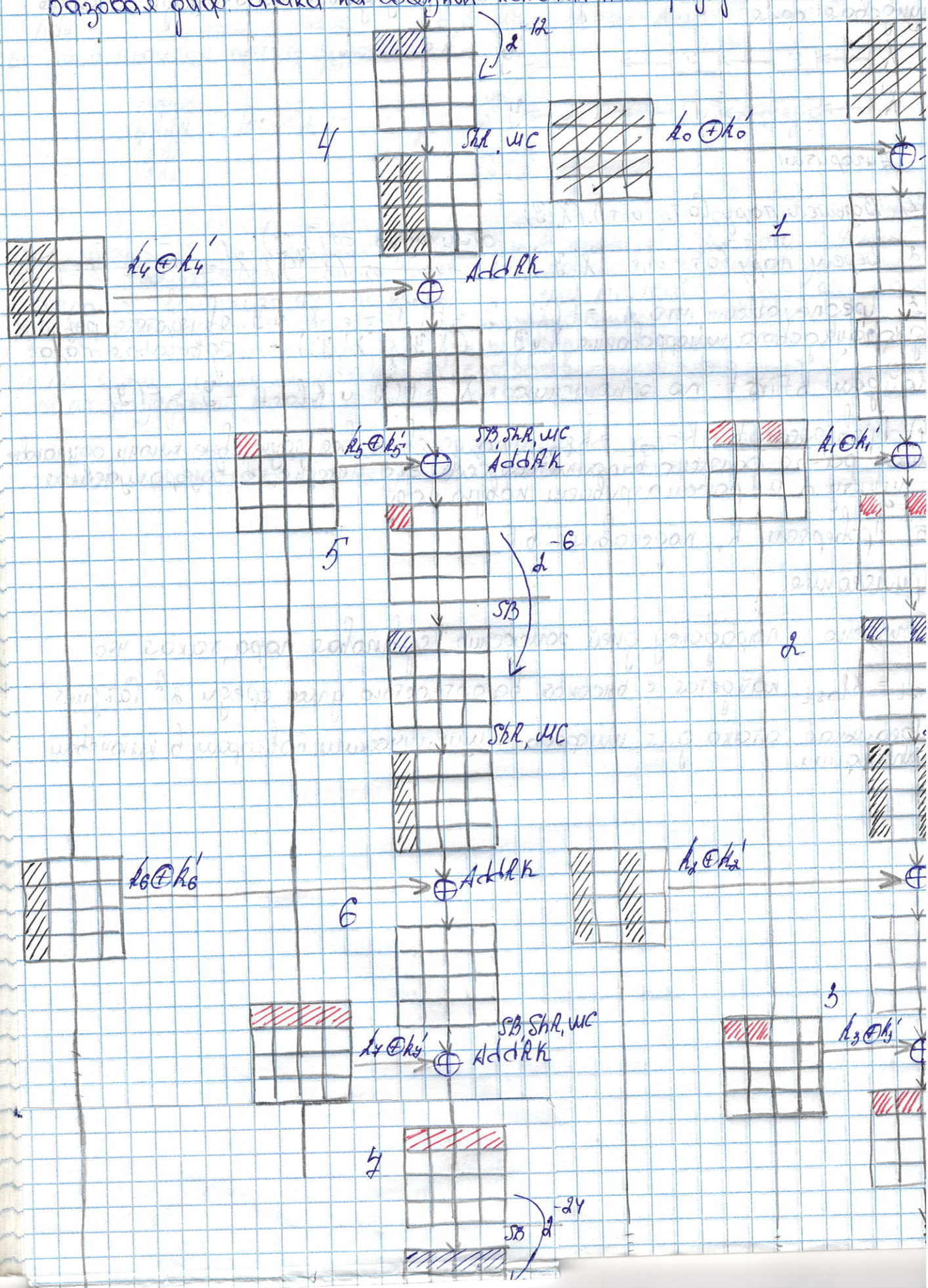
Примечание:

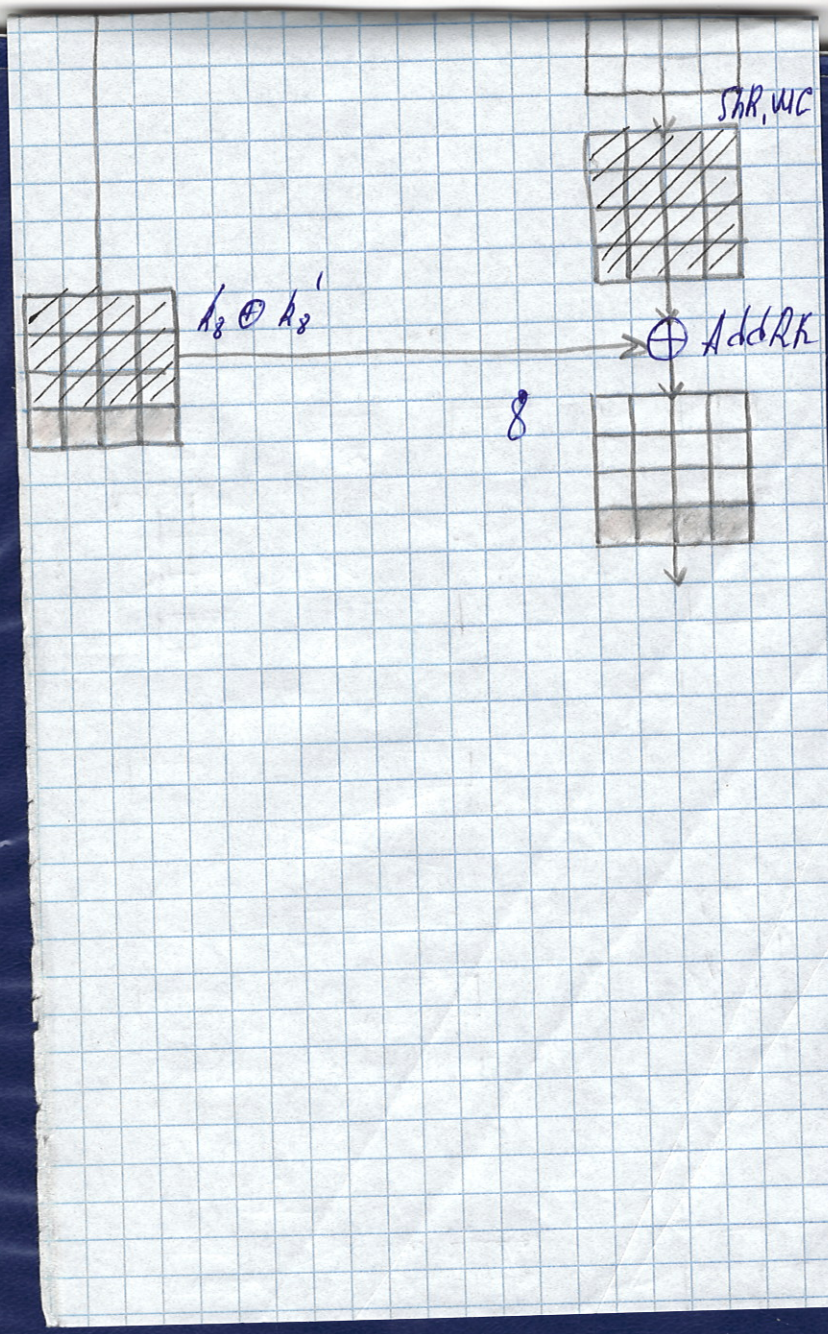
Согласно „парадоксу дней рождения" сдвиговая пара, такая, что $K_{first} = K_{last}$ найдётся с высокой вероятностью уже среди $2^{\frac{n}{2}}$ (о.т.; ш.т.)

Идеальная атака, для шифров с циклическим повтором в ключевом описании.

(18) Локальные коллизии в алгоритме AES-256. Базовая дифференциальная атака на связных ключах на 9 раундов на алгоритм AES-256

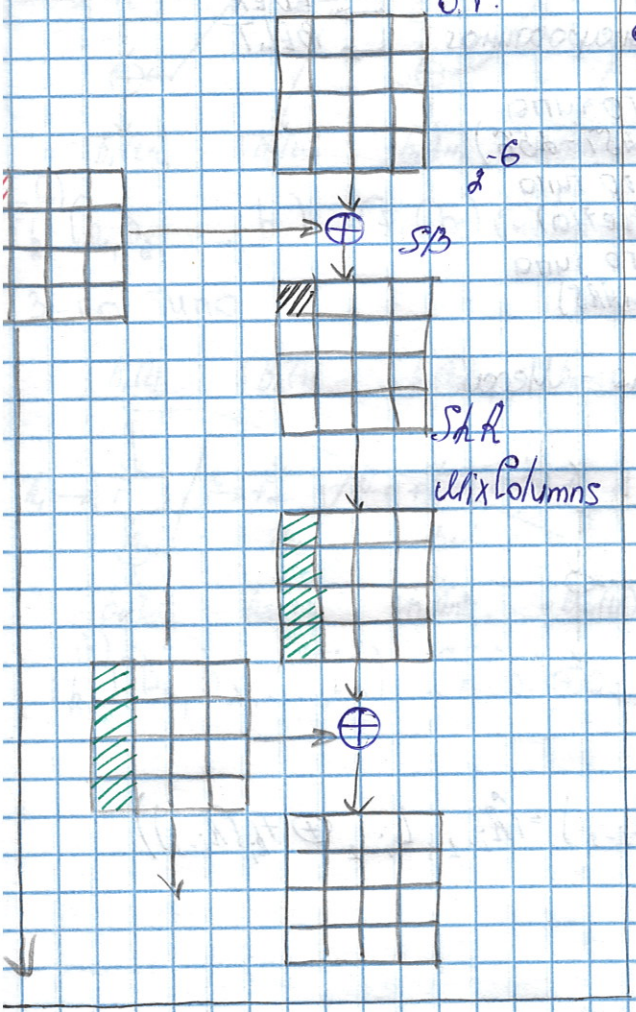
Базовая дифф. атака на связных ключах на 9 раундов на алгоритм A




ShR, WC
$k_8 \oplus k_8'$
AddRK
8

3) Локальная коллизия в алгоритме AES-256. Базовая дифференциальная атака на связных ключах на 9 раундов на алгоритм AES-256.

Локальная коллизия в алгоритме AES-256



Рассмотрим связанные ключи K и K' такие что

$K \oplus K' = (b,b,b,b,a,0,a,0)$ $a,b \in V_2^n$

они проходят следующие раундовые ключи:

$k_0 \oplus k_0' = (b,b,b,b)$

$k_1 \oplus k_1' = (a,0,a,0)$

$k_2 \oplus k_2' = (b,0,b,0)$

$k_3 \oplus k_3' = (a,a,0,0)$

$k_4 \oplus k_4' = (b,b,0,0)$

$k_5 \oplus k_5' = (a,0,0,0)$

$k_6 \oplus k_6' = (b,0,0,0)$

$k_7 \oplus k_7' = (a,a,a,a)$

$k_8 \oplus k_8' = (c,c,c,c)$

$k_9 \oplus k_9' = (d,e,d,e)$

$c = b \oplus SB(Rot(By \oplus e(a)))$

$d = SB(c) \oplus a$

$e = d \oplus a = SB(c)$

Как выбрать a и b?

Выберем произвольное ненулевое $Rot(a_0,a_1,a_2,a_3) = (a_1,a_2,a_3,a_0)$

$a \in V_2^{32}$

S-box AES обладает след. свойством:

$\forall d \in V_2^8, d \neq 0 \; \exists \beta \in V_2^8 : P(S(x \oplus d) \oplus S(x) = \beta) = 2^{-6}$ или $\left| P_x\left[S(x \oplus d) \oplus S(x) = \beta\right] - \frac{1}{4} \right|$

Таким образом $a = (d,0,0,0)^T$

$b = Mix(\beta,0,0,0)^T$

Выберем два от. P и P' $P \oplus P' = (b,b,b,b)$. Зашифруем P на ключе K и P' на K'

(19) Архитектуры блочных шифров

- Архитектуры БШ
  - SP-сеть
    - SQUARE
    - Serpent
  - Сеть Фейстеля
    - Синхронизированная (т.е. делится на две равные половины)
      - DES
      - ГОСТ
    - Несинхронизированная
      - 1-го типа (CAST-256)
      - 2-го типа (CLefia)
      - 3-го типа (MARS)
  - Лай-Мейси
    - IDEA
    - BELT

SP-сеть

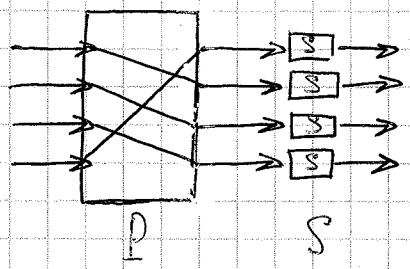


Схема Лая - Мейси

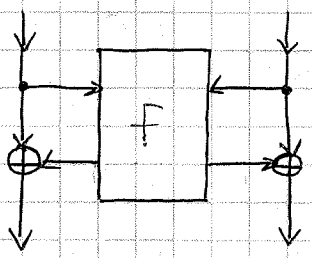
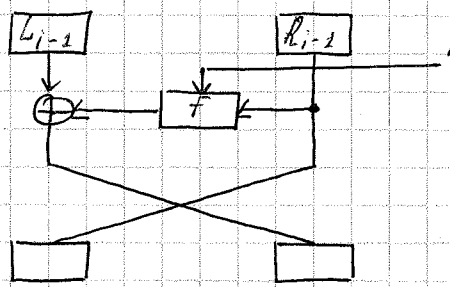


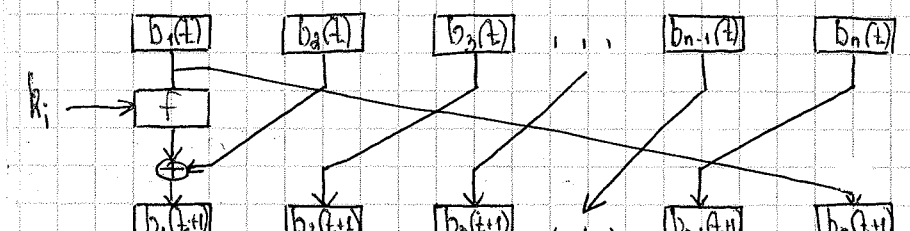
Схема Фейстеля



$$F_{k_i}(L_{i-1}, R_{i-1}) = (R_{i-1}, L_{i-1} \oplus f_{k_i}(R_{i-1}))$$

- DES, ГОСТ

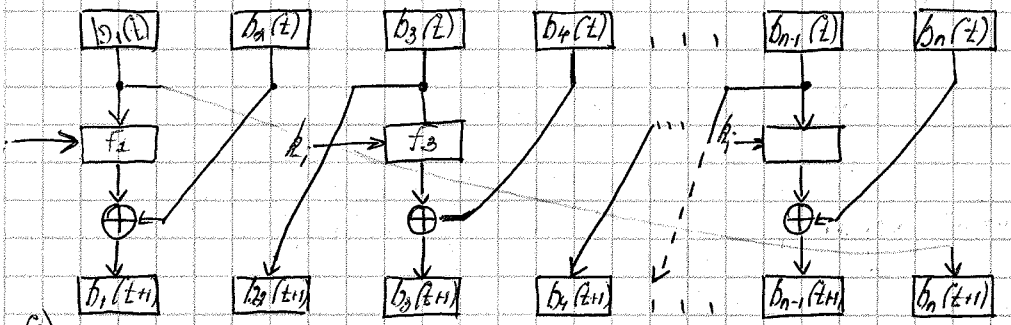
1-го типа



$$F^{(1)}_{k_i}(b_1, b_2, \dots, b_n) =$$
$$= (b_2 \oplus f_{k_i}(b_1), b_3, b_4, \dots, b_n, b_1)$$

- CAST-256

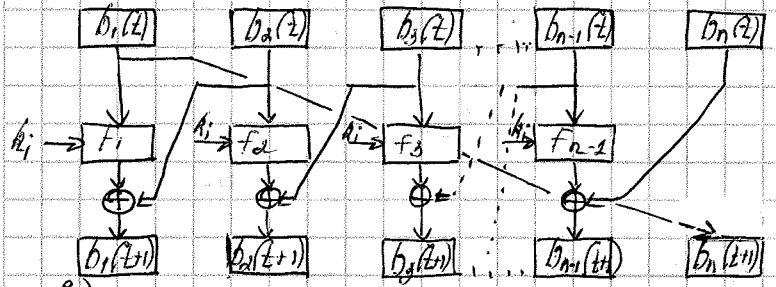
2-го типа



· CleFia

$$F_k^{(2)}(b_1, b_2, \ldots, b_n) = (f_1(b_1) \oplus b_2, b_3, f_3(b_3) \oplus b_4, \ldots, f_{n-1}(b_{n-1}) \oplus f_n, b_1)$$

3-го типа

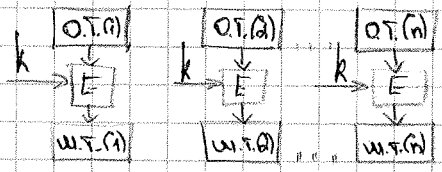


· MARS

$$F_k^{(3)}(b_1, b_2, \ldots, b_n) = (f_1(b_1) \oplus b_2, f_2(b_2) \oplus b_3, \ldots, f_{n-1}(b_{n-1}) \oplus b_n, b_1)$$

Режимы шифрования блочных шифров

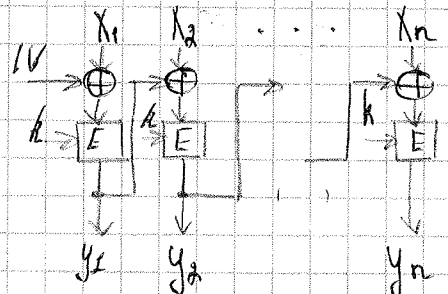
ECB - Electronic Code Book



$ш.т.(i) = E(k, о.т.(i))$

Сохранение статистических особенностей т.к. если $y_i = y_j$, $i \neq j$, то $D(y_i, k) = D(y_j, k)$

CBC - Cipher Block Chaining



$y_1 = E(IV \oplus x_1, k)$

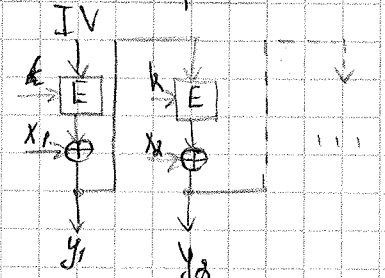
$y_j = E(x_i \oplus y_{j-1}, k)$, $i = \overline{2,n}$

$y_i = y_j \Leftrightarrow E(x_i \oplus y_{i-1}, k) = E(x_j \oplus y_{j-1}, k)$

$x_i \oplus y_{i-1} = x_j \oplus y_{j-1}$

$x_i \oplus x_j = y_{i-1} \oplus y_{j-1}$

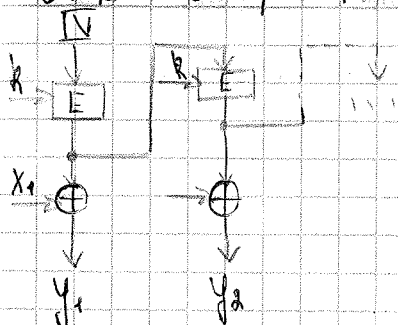
CFB - Cipher Feed Back



$y_1 = E(IV, k) \oplus x_1$

$y_i = E(y_{i-1}, k) \oplus x_i$, $i = \overline{2,n}$

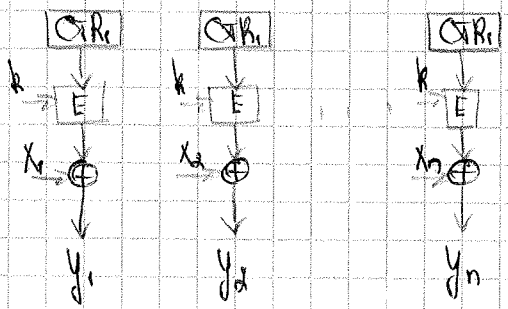
OFB - Output Feed Back



$y_1 = E(IV, K) \oplus X_1 = r_1 \oplus x_1$

$y_i = E(E(y_{i-1}, k), k) \oplus x_i = E(r_{i-1}, k) \oplus$

CTR - Counter mode

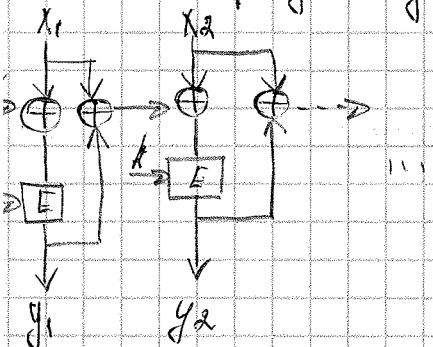


$y_i = x_i \oplus E(CTR_i, k)$ $i = \overline{1,n}$

$CTR_i = \underbrace{Nonce}_{fix}\ \underbrace{CounterValue}_{изменяемая\ часть}$, где $i = \overline{1,n}$

- Nonce - уникальное значение, задаваемое раз при шифровании всего сообщения. Может быть случайным или заранее определенным. !Не должен повторяться при одном и том же ключе!
- CounterValue - изменяемая часть CTR, увеличивается от блока к блоку, обычно на 1, обеспечивает уникальность входа в блочный шифр для каждого блока.

PCBC - Propagating Cipher Block Chaining



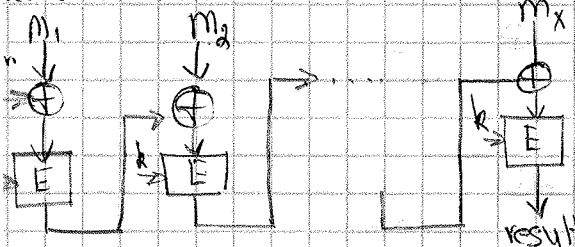
Изменение блока м.т. приводит к неправильному дешифрованию всех последующих блоков, тем самым обеспечивая целостность сообщения.

$y_1 = E(x_1 \oplus IV, k)$

$y_i = E(x_i \oplus x_{i-1} \oplus y_{i-1}, k)$, $i = \overline{2,n}$

Аутентификация сообщений. CBC-MAC(Cipher Block Chaining - essage Autentification Code)

утентификация сообщений - механизм который позволяет подтвердить целостность данных, подтвердить подлинность источника. Для этого ользуют MAC.



m_x $E: K_E \times \{0,1\}^n \to \{0,1\}^n$, где $E(K,x) = E_k(x)$ является перестановкой $\{0,1\}^n$, K_E - набор вероятностных ключей, n - длина блока.

Обозначаем $X_0 = IV = 0^n$, тогда $X_i = E(m_i \oplus X_{i-1})$ - промежуточное значение после i-го блока $i = \overline{1,x}$ result $= X_x$

промежуточное значение после i-го блока

Дополнение последнего блока в стандартах!

Дополнение битами: „1" + „0" + „0" + ... + „0" - MD5, SHA

байтами!

дополнение нулями с индикатором длины!

- Если для дополнения блока необходимо добавить, например, 4 байта, то в конец сообщения добавляются 4 байта padding. При этом добавляемые байты, кроме последнего, равны нулю. Последний байт хранит кол-во добавляемых байт (04). | 00 00 00 00 00 00 00 04 |

дополнили

) Случайные байты, а последний - кол-во добавленных

| DD DD DD DD 06 80 99 04 |

Значение байтов - кол-во дополненных: |01|
|02|02|
|03 03 03|

добавление 0x80 и 00...00 до нужной длины.

(21) Методы объединения блочных шифров

Двойное шифрование

$C = E_{k_2}(E_{k_1}(P))$! Не эффективно, т.к. подвержено атаке MITM

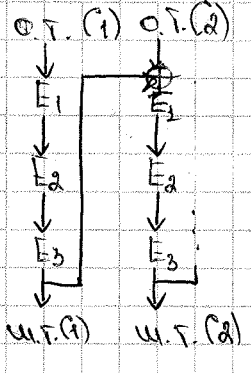
$P = D_{k_1}(D_{k_2}(C))$

Тройное шифрование

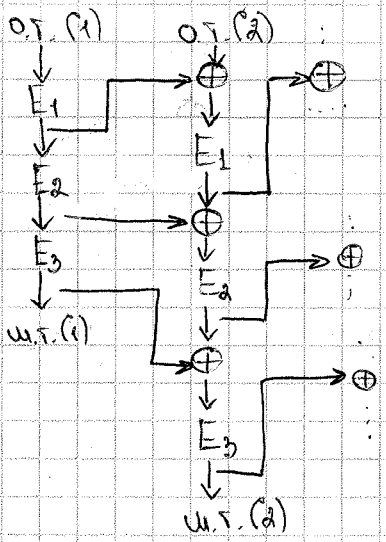
$C = E_{k_1}(D_{k_2}(E_{k_1}(P)))$! Эффективно

$P = D_{k_1}(E_{k_2}(D_{k_1}(C)))$

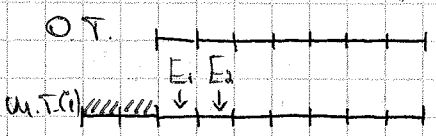
Внешний СВЗС



Внутренний СВЗС



Шифрование с дополнением



8 Проблема распределения ключей. Протокол Диффи-Хеллмана. Метод „человек в середине". Защита от атаки „человек в середине". Трехсторонний протокол Диффи-Хеллмана.

Если имеется n абонентов и нужно наладить между ними шифрованную связь, то кол-во ключей C_n^2

Протокол Диффи-Хеллмана

A,B выбирают некоторое $p \in \mathbb{P}$ - большое простое число и получают $\mathbb{Z}_p^* = \{1, 2, \dots, p-1\}$ - мультипликативная группа вычетов, она циклическая. Обычно выбирают α - образующий эл-т $\mathbb{Z}_p^*$

1) A выбирает случайно $x \in [1; p-2]$ $\alpha^x \bmod p \to B$
2) B выбирает $y \in [1; p-2]$ и $\alpha^y \bmod p \to A$
3) A: $(\alpha^y \bmod p)^x \bmod p = \alpha^{xy} \bmod p$ \ Общий
   B: $(\alpha^x \bmod p)^y \bmod p = \alpha^{xy} \bmod p$ — секретный ключ

C (криптоаналитик) знает: $\alpha^x, \alpha^y, \alpha, p$.

Стойкость основана на сложности дискретного логарифмирования (криптоаналитик знает α и p, то ему нужно найти такой x, что $\alpha^x \bmod p$ - трудная задача)

Метод „человек по середине"

1) $A \xrightarrow{\alpha^x \bmod p} C \xrightarrow{\alpha^{x'} \bmod p} B$
2) $A \xleftarrow{\alpha^{y'} \bmod p} C \xleftarrow{\alpha^y \bmod p} B$
3) $A \xleftrightarrow{\alpha^{xy'} \bmod p} C \xleftrightarrow{\alpha^{x'y} \bmod p} B$

x' и y' C генерирует сам. Защита: использование ЭЦП

Трехсторонний протокол Диффи-Хеллмана

1) A $\xrightarrow{\alpha^x}$ B; A $\xleftarrow{\alpha^z}$ D; B $\xrightarrow{\alpha^y}$ D
2) A $\xrightarrow{\alpha^{zx}}$ B; A $\xleftarrow{\alpha^{zy}}$ D; B $\xrightarrow{\alpha^{xy}}$ D

Общий секретный ключ $= \alpha^{xyz} \bmod p$

(23) Протоколы STS, MTI. Трехпроходный алгоритм Шамира.

STS. Station-to-station

Протокол предполагает, что пользователи применяют цифровую подпись, которая дописывается пересылаемые по протоколу Диффи-Хеллмана сообщения:

1) $A \to B: \alpha^x \bmod p$

2) $B \to A: \alpha^y \bmod p, E_k(S_B(\alpha^y, \alpha^x))$

3) $A \to B: E_k(S_A(\alpha^x, \alpha^y))$

Здесь S_A и S_B – цифровые подписи пользователей A и B соответственно, k – искомый общий ключ. Они позволяют гарантировать достоверность полученное сообщение именно от того пользователя, от которого сообщение получено. Шифрование значений подписей пользователей введено для того, чтобы обеспечить взаимное подтверждение правильности вычисления значения ключа.

MTI

Предположим, что пользователи A и B имеют секретные ключи a, $1 \le a \le p-2$, и b, $1 \le b \le p-2$ соответственно и публикуют свои открытые ключи $Z_A = \alpha^a \bmod p$ и $Z_B = \alpha^b \bmod p$. Для выработки общего секретного ключа k они должны сгенерировать случайные числа X, $1 \le X \le p-2$, и Y, $1 \le Y \le p-2$, соответственно, а затем обменяться следующими сообщениями:

1) $A \to B: \alpha^X \bmod p$

2) $B \to A: \alpha^Y \bmod p$

Искомый общий ключ вычисляется по формуле:

$$k = (\alpha^Y)^a Z_B^X = (\alpha^X)^b Z_A^Y = \alpha^{Xb+Ya} \bmod p$$

! Любая подмена приведёт к тому, что стороны получат различные значения ключа.

Трехсторонний алгоритм Шамира

Данный протокол позволяет абонентам A и B безопасно обмениваться шифр-цией, не используя предварительного обмена ни с.к. ни о.к. Он предполагает использование коммутативного симметрического преобразования E для которого: $E_A(E_B(P)) = E_B(E_A(P))$

Алгоритм:

1) $A \to B: E_{kA}(k)$

2) $B \to A: E_{kB}(E_{kA}(k))$

3) $A \to B: D_{kA}(E_{kB}(E_{kA}(k)))$

Заметим, что в этом протоколе можно использовать не каждое коммутативное преобразование E.

Например, для $E_{kA}(k) = k \oplus \Gamma$ протокол явно неспособен

□. $E_{kA}(k) = k \oplus \Gamma_1$, $E_{kB}(E_{kA}(k)) = E_{kB}(k \oplus \Gamma_1) = k \oplus \Gamma_1 \oplus \Gamma_2$

$D_{kA}(E_{kB}(E_{kA}(k))) = k \oplus \Gamma_2$ | A отпр. B $k \oplus \Gamma_1$. C перехватывает, тогда

$B \to A$ $k \oplus \Gamma_1 \oplus \Gamma_2$. C может вычислить $\Gamma_2(k \oplus \Gamma_1 \oplus \Gamma_2 \oplus k \oplus \Gamma_1) = \Gamma_2$

когда $A \to B$ $k \oplus \Gamma_2$ $C \oplus$ result с Γ_2 и получает k ☒

Поэтому в протоколе Шамира рекомендуется использовать преобразование вида $E_{k_A}(k) = k^a \bmod p$, в котором константа a определяется ключом k_A.

(24) Определение шифра с открытым ключом. Задача укладки рюкзака. Криптосистема Меркля-Хеллмана.

Опр. (X, K, Y, E, D) – криптосистема с открытым ключом, если выполняется: $K = (k_0, k_c)$, $E_k = E_{k_0}$, $D_k = D_{k_c}$

1) $\forall x \in X \; \forall k \in K \; D_k(E_k(x)) = x$
2) Зная k_0 сложно восстановить k_c (NP-трудная задача)
3) E_{k_0}, D_{k_c} – полиномиальные алгоритмы (P-задача)
4) $\exists$ полиномиальный алгоритм генерации ключей

! Замечание

Если использовать криптосистему с о.к. для ЭЦП то требуется выполн. условие к П.1: $\forall y \in Y \; \forall k \in K \; E_k(D_k(y)) = y \Rightarrow$ ЭЦП $(x, D_{k_c}(x))$
Проверка $E_{k_0}(D_{k_c}(x)) = x$

Криптосистема Меркля – Хеллмана

Основывается на NP-трудной задаче укладки рюкзака.

$\exists a_i \in \mathbb{Z}$ – веса предметов $x_i \in \{0, 1\}$ – индикатор наличия предметов $i = \overline{1,n}$, $a_1 x_1 + \ldots + a_n x_n = S$ – вес рюкзака.

Задача: дан набор весов $(a_1, \ldots, a_n)$, S. Найти $\vec{x}$

Решение: Перебор $\vec{x}$ и проверка $\sum_{i=1}^{n} a_i x_i = S$ $O(2^n)$

Но $\exists \{a_i\}$ для которых вычисление $\vec{x}$ по S не является сложной (NP-трудной)

Опр. Послед-ть $\{a_i\}_{i=1}^{n}$ наз-ся супервозрастающей, если $\forall j \in \overline{2,n} \; a_j > a_1 + \ldots + a_{j-1}$ Пример $\{2, 5, 8, 17, \ldots\}$

Алгоритм

A: выбирает супервозрастающую посл-ть $\vec{a} = (a_1, \ldots, a_n)$
выбирает $m \in \mathbb{N}$ $m > \sum_{i=1}^{n} a_i$ и $\omega \in \mathbb{N}$: $(\omega, m) = 1$.
Каждый $a_i \times \omega \pmod m$ и получается $\vec{b} = (b_1, \ldots, b_n)$ $\vec{b}$ – о.к., $\vec{a}, m, \omega$ – с.к.

B: x – сообщение $x \in V_2^n$. Вычисляет $\sum_{i=1}^{n} b_i x_i = S$ и посылает A

A: Расшифровывает $S' = S \cdot \omega^{-1} \pmod m$ по супервозрастающей по-ти вычисляет $\vec{x}$.

Пример:

Дано:

$\{a\} = (3, 5, 11, 23)$

$\sum_{i=1}^{n} a_i = 42 \Rightarrow m = 43$

$\omega = 20, X = (1010)$

$b - ?\ S - ?\ y - ?$

Решение:

$\vec{b} = (b_1, b_2, b_3, b_4)$

$b_1 = (20 \cdot 3) \bmod 43 = 17$

$b_2 = (20 \cdot 5) \bmod 43 = 14$

$b_3 = (20 \cdot 11) \bmod 43 = 5$

$b_4 = (20 \cdot 23) \bmod 43 = 30$

$X = (1010)\quad S = 17 \cdot 1 + 14 \cdot 0 + 5 \cdot 1 + 30 \cdot 0 = 22$

$\omega^{-1} = 20^{-1} = 28 \pmod{43}$

$S^1 = S \cdot \omega^{-1} \bmod m = 22 \cdot 28 \bmod 43 = 616 \equiv 14 m$

$14 = x_1 \cdot 3 + x_2 \cdot 5 + x_3 \cdot 11 + x_4 \cdot 23 \Rightarrow x_1 = 1, x_2 = 0, x_3 =$

$\Rightarrow (1010)$

Ответ: $(17, 14, 5, 30)$; 22; (1010).

(25) RSA. Доказательство корректности работы RSA.

Криптосистема основана на задаче факторизации.

I p, q - различные БОЛЬШИЕ простые числа

$N = p \cdot q$ — модуль криптосистемы

$\varphi(N) = (p-1)(q-1)$ - ф-я Эйлера

Пользователь выбирает e - показатель зашифрования (о.к.)

$1 < e < \varphi(N)$

$(e, \varphi(N)) = 1$

Тогда $\exists! d : 1 < d < \varphi(N) : ed \equiv 1 \bmod \varphi(N)$

d - показатель расшифрования

Алгебраическая модель RSA

$\sum_{RSA} = (X, K, Y, E, D)$, где $X = Y = \mathbb{Z}_N$

$K = (N, q, p, e, d)$

$\{N, e\}$ - открытый ключ

d - секретный ключ

p, q - не нужны но должны храниться в секрете

$E_k(x) = E_{(N,e)}(x) = x^e \bmod N$

$D_k(y) = D_d(y) = y^d \bmod N$

Корректность работы RSA

Теорема $\forall x \in \mathbb{Z}_N, D_k(E_k(x)) = x$, т.е. $x^{ed} \equiv x \bmod N$

□ Малая т-ма Ферма. Если $(a, p) = 1$, то $a^{p-1} \equiv 1 \bmod p$

Обобщение Эйлера т-мы Ферма. Если $(a, m) = 1$, то $a^{\varphi(m)} \equiv 1 \bmod m$

1) Рассмотрим случай, когда $(x, N) = 1$

$e \cdot d \equiv 1 \pmod{\varphi(N)} \Rightarrow ed - 1 = k\varphi(N) \Rightarrow ed = k \cdot \varphi(N) + 1$

$x^{ed} = x^{k \cdot \varphi(N) + 1} = (x^{\varphi(N)})^k \cdot x \equiv x \bmod N$

а) Рассмотрим случай, $(x, N) = p$, т.е. $p | x$ и $(x, q) = 1$, т.е. $q \nmid x$

Тогда $x = b \cdot p$, $b \in \mathbb{N} \Rightarrow x \bmod p \equiv 0 \bmod p$ (1), следовательно

$x^{ed} = bp^{ed} \Rightarrow x^{ed} \bmod p \equiv bp^{ed} \bmod p \equiv 0 (\bmod p)$ (2)

Из (1), (2) следует, что $x^{ed} \equiv x \bmod p$ (3)

Т.к. $(x, q) = 1$, тогда $x^{\varphi(q)} \equiv 1 \bmod q$. То $x^{ed} \bmod q \equiv (x^{ed-1}) x \bmod q$

$\equiv x^{\varphi(N) \cdot e} \cdot x \bmod q \equiv x^{(q-1)(p-1)e} \cdot x \bmod q \equiv x^{(q-1)(p-1)e} \cdot x \bmod q \equiv 1^{(p-1)e} \cdot x$

$\equiv x \bmod q$ (4), где $c \in \mathbb{N}$

Из (3), (4) следует, что $x^{ed} \equiv x \bmod N$

б) Рассмотрим случай $(x, N) = q$, т.е $q | x$ и $(x, p) = 1$, т.е $p \nmid x$

Тогда $x = a \cdot q$, $a \in \mathbb{N} \Rightarrow x \bmod q \equiv 0 \bmod q$ (1)

$x^{ed} = aq^{ed} \Rightarrow x^{ed} \bmod q \equiv 0 \bmod q$ (2)

Из (1), (2) следует, что $x^{ed} \bmod q \equiv x \bmod q$ (3)

Т.к. $(x, p) = 1$, тогда $x^{\varphi(p)} \equiv 1 \bmod p$. То $x^{ed} \bmod p \equiv (x^{ed-1}) x \bmod p \equiv$

$\equiv x^{\varphi(N) \cdot f} \cdot x \bmod p \equiv x^{(p-1)(q-1)f} \cdot x \bmod p \equiv 1^{(q-1)f} \cdot x \bmod p \equiv x \bmod p$ (4)

Из (3), (4) следует, что $x^{ed} \equiv x \bmod N$

№6 Задачи RSAFP, RSADP, PSAKRP, RSAOP и их полиномиальная сводимость друг к другу.

RSAFP (RSA Factorization problem)

Задача факторизации модуля RSA

Дано: число N - модуль криптосистемы RSA

Найти: разложение числа N на p и q

RSADP (RSA Decryption problem)

Задача раскрытия системы RSA

Дано: открытый ключ (N, e) и ш.т. полученный на нём

Найти: x из $x^e \pmod N = y$

RSAKRP (RSA Key Recovery Problem)

Задача нахождения секретного ключа RSA

Дано: открытый ключ (N, e)

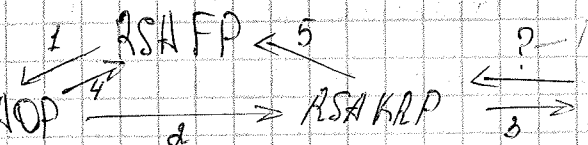
Найти: секретный ключ d: $d \cdot e \equiv 1 \bmod \varphi(N)$

RSAOP (RSA Order problem)

Задача нахождения порядка мультипликативной группы RSA

Дано: число N - модуль криптосистемы RSA

Найти: число $\varphi(N)$ (т.е. порядок мультипликативной группы $\mathbb{Z}_N^*$)



1) $FACT(N) = p \cdot q \Rightarrow \varphi(N) = (p-1)(q-1)$

2) Из $\varphi(N)$ и $(N, e) \Rightarrow d \cdot e \equiv 1 \bmod \varphi(N)$

3) Из d и (N, e) $(x^e)^d \equiv x \bmod N \Rightarrow x$

4) Из $\varphi(N)$ и $N \Rightarrow N = p \cdot q$

$$\begin{cases} p \cdot q = N \\ (p-1)(q-1) = \varphi(N) \end{cases} \quad \begin{cases} q = \frac{N}{p} \\ (p-1)\left(\frac{N}{p} - 1\right) = \varphi(N) \end{cases} \quad \begin{cases} q = \frac{N}{p} \\ N - \frac{N}{p} - p + 1 = \varphi(N) \end{cases}$$

$$\begin{cases} q = \frac{N}{p} \\ p^2 - pN + N - p = \varphi(N) \cdot (-p) \end{cases}$$

5) Из $d \Rightarrow ed \equiv 1 \bmod \varphi(N) \Rightarrow$

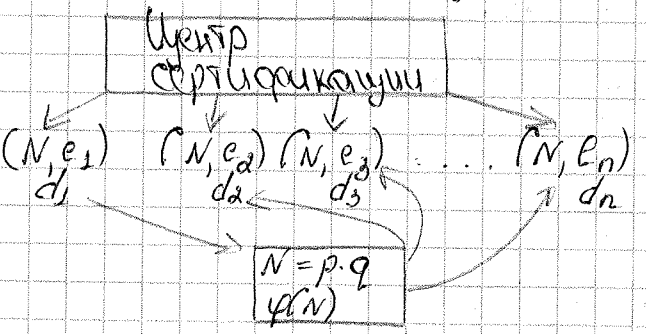
$\Rightarrow ed - 1 = k \cdot \varphi(N) \Rightarrow$

$\Rightarrow \varphi(N) = \frac{ed-1}{k} \Rightarrow p \cdot q$

(Т полиномиальный вер-й алгоритм)

(27) Слабости RSA

1) Использование общего модуля криптосистемы (N)

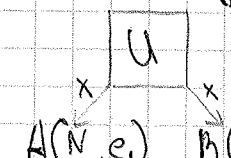


Проблема: Зная (N, e_1) и d_1 можно вычислить $\varphi(N)$, а значит можно вычислить $d_2, d_3, \dots, d_n$. Следовательно взлом одного приводит к взлому всех участников сети.

2) Рассылка циркуляра на общем модуле криптосистемы

Определение

Циркуляр – сообщение, направляемое одним пользователем многим адресатам.



Проблема: $\begin{cases} y_1 \equiv x^{e_1} \bmod N \\ y_2 \equiv x^{e_2} \bmod N \end{cases}$

Т. к. $(e_1, e_2) = 1 \Rightarrow \exists U, V \in \mathbb{Z} : e_1 U + e_2 V = 1 \Rightarrow$

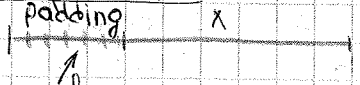
$\Rightarrow y_1^U \cdot y_2^V = x^{e_1 U + e_2 V} = x$ (вскрывается чтение)

3) Использование малой экспоненты (e)

Если $x < [\sqrt[e]{N}]$, то при зашифровании не происходит приведение по модулю

$y \equiv x^e \bmod N = x^e \Rightarrow x = \sqrt[e]{y}$ – легко вычисляется

Исправление



набивка помогает увеличить x, чтоб происходило приведение по модулю

Гомоморфный шифр

Опр. Шифр $\Sigma_A = (X, K, Y, E, D)$ – гомоморфный, если:

$$\begin{cases} D(E(x_1) \cdot E(x_2)) = x_1 \cdot x_2 \\ D(E(x_1) + E(x_2)) = x_1 + x_2 \end{cases} \quad \forall x_1, x_2 \in X$$

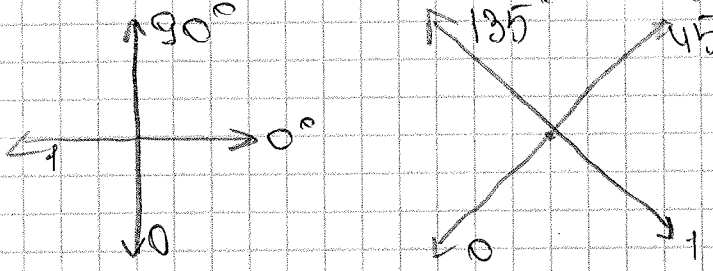
Частичная гомоморфность RSA

RSA - частично гомоморфен по умножению

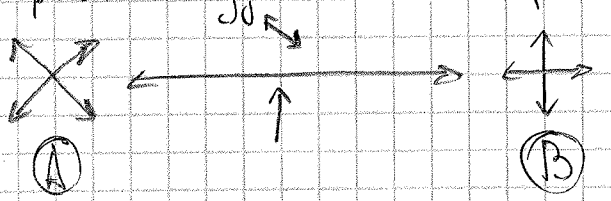
$$D(E(x_1)\ E(x_2)) = (x_1^e \bmod N \cdot x_2^e \bmod N)^d = x_1^{ed} \bmod N \cdot x_2^{ed} \bmod N = x_1 \cdot x_2$$

3 Протокол BB84

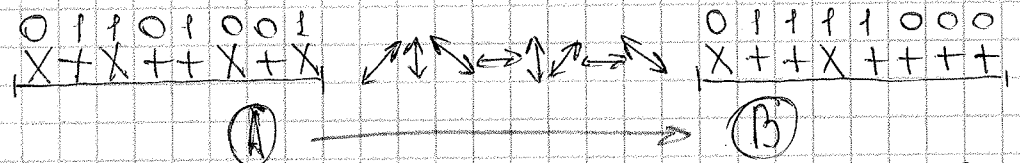
В данном протоколе A будет пересылать B единичные кубиты с определенной поляризацией. Используются 4 квантовых состояния ($0°, 45°, 90°, 135°$) в 2 базисах.



При передаче каждого бита сообщения A случайным образом выбирает базис. В таким же случайным образом выбирает базис, в котором он будет этот бит принимать.



В таком случае, B с вер-ю $\frac{1}{2}$ будет принимать правильный бит.



После передачи сообщения A и B обмениваются по открытому каналу связи информацией о том, в каком базисе они отправили и принимали сообщение. Зная это, они могут исключить из сообщения те биты, при передаче которых были выбраны разные базисы, и получить т.о. одинаковый ключ.